



Dataflow & Behavioral Modeling



BITS Pilani
Pilani Campus

Prof. GSS Chalapathi,
Prof. Saurabh Gandhi,
Prof. Ramesh Sambangi

Data Types in Verilog HDL

Nets:

- ❖ Nets represents a physical connection
- ❖ Does not store a value
- ❖ Driven continuously by a source
- ❖ Declared using keyword `wire`.

Registers:

- ❖ Can hold a value until it is assigned again
- ❖ Declared using keyword `reg`.
- ❖ Used for signals assigned inside initial or always blocks
- ❖ `reg` does not necessarily mean a hardware register

`integer` → signed 32-bit integer, commonly used in testbenches

`time` → stores simulation time

`real` → floating-point value, mainly for simulation

Data types contd..

integer → signed 32-bit integer, commonly used in testbenches

Ex: integer count;

```
initial begin
    count = 10;
end
```

time → stores simulation time

```
time t;
initial begin
    #10 t = $time;
    $display("Time = %t", t);
end
```

real → floating-point value, mainly for simulation

```
real voltage;
initial begin
    voltage = 1.25;
end
```

Data types contd..

Vectors: A vector represents multiple bits as a single signal.

```
wire [3:0] A;
```

Arrays: An array is a collection of elements.

```
reg [7:0] data [0:3];
```

```
integer count[0:7]; // An array of 8 count variables
reg bool[31:0]; // Array of 32 one-bit boolean register variables
time chk_point[1:100]; // Array of 100 time checkpoint variables
reg [4:0] port_id[0:7]; // Array of 8 port_ids; each port_id is 5 bits wide
integer matrix[4:0][4:0]; // Illegal declaration. Multidimensional
array
```

Memory: A memory can be represented as an **array of vectors**.
useful for representing RAM, ROM, register files, lookup tables,
instruction memory, and data memory.

```
reg [7:0] memory [0:15];
```

Data types contd..

Parameters: A parameter represents a constant that can be used to make a design configurable.

```
parameter WIDTH = 8;  
wire [WIDTH-1:0] A;
```

localparam: is also a constant, but it cannot normally be overridden during module

```
localparam IDLE = 2'b00;  
localparam RUN  = 2'b01;  
localparam DONE = 2'b10;
```

Data types contd..

String: Verilog does not have a separate string data type in traditional Verilog. A string can be stored in a register/vector.

```
reg [11*5-1:0] var_name;  
initial begin  
    var_name = "BITS PILANI";  
end
```

System Tasks

System tasks are **built-in Verilog commands** provided by the simulator.

Task	Function	Example
<code>\$display</code>	Display once	<code>\$display("A=%b",A);</code>
<code>\$write</code>	Display without newline	<code>\$write("A=%b",A);</code>
<code>\$monitor</code>	Continuously monitor changes	<code>\$monitor("A=%b",A);</code>
<code>\$time</code>	Current simulation time	<code>\$time</code>
<code>\$stop</code>	Suspend simulation	<code>\$stop;</code>
<code>\$finish</code>	End simulation	<code>\$finish;</code>

\$display statement

- Used to print values and messages during simulation.
- Prints only when the statement is executed.

```
reg a, b;  
reg [3:0] sum;  
initial begin  
    a = 1;  
    #10 b = 0;  
    sum = 4'b1010;  
    $display("a = %b, b = %b, sum = %b", a, b, sum);  
end
```

output:

```
a = 1, b = 0, sum = 1010
```

String format



Format	Display
%d or %D	Display variable in decimal
%b or %B	Display variable in binary
%s or %S	Display string
%h or %H	Display variable in hex
%c or %C	Display ASCII character
%m or %M	Display hierarchical name (no argument required)
%v or %V	Display strength
%o or %O	Display variable in octal
%t or %T	Display in current time format
%e or %E	Display real number in scientific format (e.g., 3e10)
%f or %F	Display real number in decimal format (e.g., 2.13)
%g or %G	Display real number in scientific or decimal, whichever is shorter

\$monitor



- keeps watching signals and prints whenever their values change.

Ex:

```
reg a, b;
initial begin
    $monitor("Time=%0t : a=%b b=%b", $time, a, b);
    a = 0;
    b = 0;
    #10 a = 1;
    #10 b = 1;
End
```

Output:

```
Time=0    : a=0 b=0
Time=10   : a=1 b=0
Time=20   : a=1 b=1
```

\$stop & \$finish

When the simulator encounters \$stop:

- The simulation is paused.
- The simulator enters a debug/interactive mode.
- You can inspect signal values, waveforms, and variables.
- The simulation can generally be resumed from that point.

When the simulator encounters \$finish:

- The current simulation is terminated.
- The simulator exits the simulation run.
- No further Verilog statements are executed.
- It is commonly used at the end of a testbench after all test cases are completed.

DATAFLOW MODELING IN VERILOG

Dataflow modeling describes the digital circuit in terms of **data flow between inputs and outputs** using **continuous assignments** and operators.

1. WHAT IS DATAFLOW MODELING?

- Specifies **how** data flows from inputs to outputs using operators and continuous assignments.
- No description of internal gates or procedural statements.
- Suitable for representing **combinational logic**.
- Uses **assign** statement.

General Form:

```
assign output_signal = expression;
```

2. OPERATORS USED

Operator	Meaning	Example	Result
&	AND	a & b	a AND b
	OR	a b	a OR b
^	XOR	a ^ b	a XOR b
~	NOT	~a	NOT a
^~	XNOR	a ^~ b	a XNOR b
~&	NAND	a ~& b	NAND
~	NOR	a ~ b	NOR

3. EXAMPLE : 2 – INPUT XOR GATE

Boolean Expression:

$$Y = A \oplus B$$

Logic Diagram:



Dataflow Model (Verilog):

```
module xor_dataflow (
    input A, B,
    output Y
);
    assign Y = A ^ B;
endmodule
```

4. EXAMPLE : 2 – INPUT AND – OR GATE

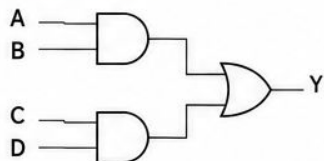
Boolean Expression:

$$Y = (A \& B) | (C \& D)$$

Dataflow Model (Verilog):

```
module and_or_dataflow (
    input A, B, C, D,
    output Y
);
    assign Y = (A & B) | (C & D);
endmodule
```

Logic Diagram:



- Operators follow Verilog precedence rules.
- Parentheses can be used to make the expression clear.

5. MULTI – BIT EXAMPLE

Example: 4-bit Adder (Sum Only)

$$SUM = A + B$$

Dataflow Model (Verilog):

```
module adder4_dataflow (
    input [3:0] A, B,
    output [3:0] SUM
);
    assign SUM = A + B;
endmodule
```

- Verilog supports arithmetic operators (+, -, *, /, %).

6. DATAFLOW vs GATE-LEVEL vs BEHAVIORAL

Feature	Dataflow Modeling	Gate-Level Modeling	Behavioral Modeling
Describes	Data flow using expressions	Circuit using logic gates	Algorithm / Functionality
Abstraction Level	Medium	Low	High
Implementation Detail	No gates, just expressions	Gates and connections	No hardware details
Best For	Combinational circuits	Technology mapping	Complex algorithms, FSMs
Synthesis	Yes	Yes	Yes



KEY TAKEAWAY

Dataflow modeling is a powerful and **concise** way to model **combinational** circuits in Verilog using **expressions** and **continuous assignments**.

Operator types

	Operator	Description	Example
Arithmetic Operators	+	Addition	assign sum = a + b;
	-	Subtraction	assign diff = a - b;
	*	Multiplication	assign prod = a * b;
	/	Division	assign quot = a / b;
	%	Modulus	assign rem = a % b;
Relational Operartors	==	Equal to	assign equal = (a == b);
	!=	Not equal to	assign not_equal = (a != b);
	<	Less than	assign less = (a < b);
	>	Greater than	assign greater = (a > b);
	<=	Less than or equal to	assign less_equal = (a <= b);
	>=	Greater than or equal to	assign greater_equal = (a >= b);
Logical Operators	&&	Logical AND	assign result = (a && b);
		Logical OR	assign result = (a b);
	!	Logical NOT	assign result = !a;
Bitwise Operators	&	Bitwise AND	assign result = a & b;
		Bitwise OR	assign result = a b;
	^	Bitwise XOR	assign result = a ^ b;
	~	Bitwise NOT	assign result = ~a;
Shift Operators	<<	Logical left shift	assign result = a << 1;
	>>	Logical right shift	assign result = a >> 1;
Concatenation Operator	{ }	Concatenation	assign result = {a, b};
Conditional Operator	? :	if else	assign result = sel? b : a;

Operator Precedence

- Operator precedence determines which operator is evaluated first when an expression contains multiple operators.
- Parentheses have the highest priority. The expression inside parentheses is evaluated first.

Operator	Symbols	Precedence
Unary	+ - ! ~	Highest
Multiply/Divide/Modulus	* / %	↓
Add / Subtract	+ -	↓
Shift	<< >>	↓
Relational	< <= > >=	↓
Equality	== != === !==	↓
Reduction	&, ~& ^ ^~ , ~	↓
Logical	&&	↓
Conditional	? :	Lowest precedence

Dataflow modelling rules

- The output of a continuous assignment should be a net, typically wire.
- The operands on the right-hand side can be registers or nets or function calls. Registers or nets can be scalars or vectors.

```
// Continuous assign. out is a net. i1 and i2 are nets.  
assign out = i1 & i2;
```

```
// Continuous assign for vector nets. addr is a 16-bit vector net  
// addr1 and addr2 are 16-bit vector registers.  
assign addr[15:0] = addr1_bits[15:0] ^ addr2_bits[15:0];
```

```
// Concatenation. Left-hand side is a concatenation of a scalar  
// net and a vector net.  
assign {c_out, sum[3:0]} = a[3:0] + b[3:0] + c_in;
```

Dataflow modelling contd..

Implicit Continuous Assignment

```
//Regular continuous assignment
wire out;
assign out = in1 & in2;
```

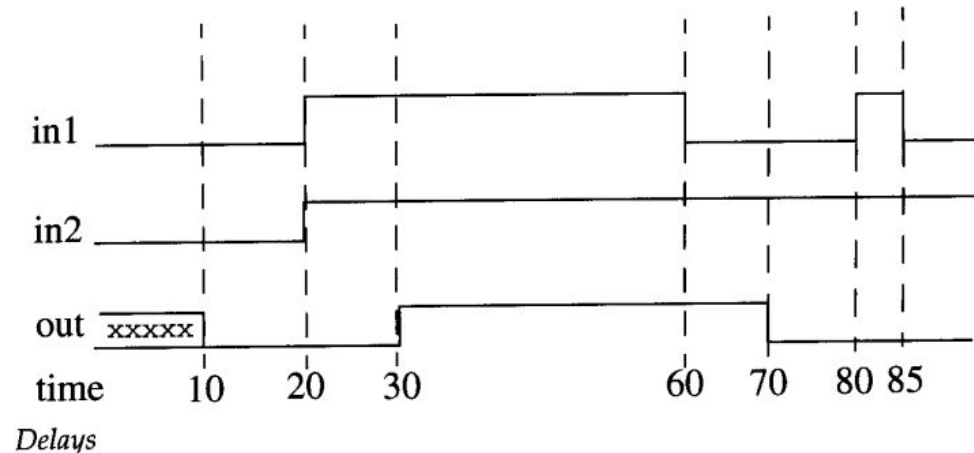
```
//Same effect is achieved by an implicit continuous assignment
wire out = in1 & in2;
```

Delays:

```
wire out;
assign #10 out = in1 & in2: // continuous assign delay
```

Implicit continuous assignment delay

```
wire #10 out = in1 & in2;
```

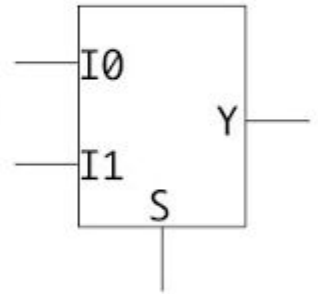


Multiplexer example

Using regular boolean expression

```

module mux2to1 (
    input  I0,I1,S,
    output Y
);
    assign Y = (~S & I0) | (S & I1);
endmodule
    
```



$$Y = \overline{S}I_0 + SI_1$$

Using conditional operator

```

module mux2to1 (
    input  I0,I1,S,
    output Y
);
    assign Y = S ? I1 : I0;
endmodule
    
```

Adder example

- RTL code specifies WHAT operation is required; synthesis determines HOW the operation is implemented in hardware.
- The synthesis tool decides the actual hardware implementation based on the target technology, constraints, and synthesis settings.

```
module adder4 (  
    input  [3:0] A,  
    input  [3:0] B,  
    input          Cin,  
    output [3:0] S,  
    output          Cout  
);  
    assign {Cout, S} = A + B + Cin;  
endmodule
```

```
module adder (  
    input  A,B,Cin,  
    output S,Cout  
);  
    assign {Cout, S} = A + B + Cin;  
endmodule
```

Behavioral Modeling

Why do we need behavioral modeling?

- Higher level of abstraction – focus on functionality rather than hardware structure.
- Less code – complex circuits can be described with a few statements.
- Easy to modify – changes in functionality are easier to implement.
- Suitable for complex digital systems – counters, controllers, processors, etc.
- Synthesizable behavioral code can be converted into hardware by synthesis tools.

Procedural Statements

- Behavioral modeling mainly uses procedural blocks: `always` and `initial`
- Other behavioral statements appear only inside these statements.
- Each `always` and `initial` statements represent separate activity flow in verilog.
- `always` and `initial` blocks cannot be nested.

Procedural block	Purpose
<code>initial</code>	Executes once at the beginning of simulation
<code>always</code>	Executes repeatedly whenever its sensitivity condition is satisfied

initial statement

- **initial** block starts at time 0, executes exactly once during a simulation.
- if multiple **initial** blocks present each block execute parallelly at 0.
- Useful for initialization monitoring

```
initial
begin
    #5 a = 1'b1;
    #25 b = 1'b0;
end
```

```
initial
begin
    #10 x = 1'b0;
    #25 y = 1'b1;
end
```

```
initial
    #50 $finish;

endmodule
```

time	statement executed
0	m = 1'b0;
5	a = 1'b1;
10	x = 1'b0;
30	b = 1'b0;
35	y = 1'b1;
50	\$finish;

always statement

- The always block is a procedural block used to describe circuit behavior.
- The block is executed repeatedly during simulation.
- Execution is controlled by a sensitivity list. The sensitivity list specifies which signal changes can trigger the always block.
- Commonly used for describing combinational and sequential circuits.
- An always block does not mean that the statements execute continuously. It repeatedly waits for an event and then executes.

```
always @(sensitivity_list) begin  
    // procedural statements  
end
```

Adder example

```
module adder4 (  
    input  [3:0] A,  
    input  [3:0] B,  
    input          Cin,  
    output reg [3:0] S,  
    output reg          Cout  
);  
    always @(A,B,Cin) begin  
        {Cout, Sum} = A + B + Cin;  
    end  
endmodule
```

Procedural Assignments

- Procedural assignments update the values of *reg*, *integer*, *real*, or *time* variables.
- The value assigned to the variable remain unchanged until another procedural update.
- Two types: blocking and nonblocking assignments

Blocking Assignments: A blocking assignment is a procedural assignment is completed before execution proceeds to the next statement. operator used '='

Nonblocking: used to schedule an update of a variable. operator used '<='

```
always @(*) begin
    Y = A & B;
    Z = Y | C;
end
```

Examples

```
initial
begin
```

```
    x = 0; y = 1; z = 1;
```

```
    count = 0;
```

```
    reg_a = 16'b0; reg_b = reg_a; //till here statements executed at 10ns
```

```
    #15 reg_a[2] = 1'b1; //this statement at executed 15ns
```

```
    #10 reg_b[15:13] = {x, y, z} //this statement executed at 25ns
```

```
    count = count + 1; //this is also executed at 25ns
```

```
end
```

Blocking

```
initial
begin
```

```
    //all 'in1','in2' and 'in3' are sampled at 0 time unit.
```

```
    //assign at 1 time unit.
```

```
    reg1 <= #1 in1;
```

```
    // assign at 10 time units
```

```
    reg2 <= #10 in2 ^ in3;
```

```
    // assign at 1 time unit
```

```
    reg3 <= #1 reg1; //The old value of reg1
```

```
end
```

Nonblocking

Other statements

- Conditional statements

```
if (condition)
    statement1;
else
    statement2;
```

- Multiway branching: case statement

```
case (expression)
    value1: statement;
    value2: statement;
    value3: statement;
    default: statement;
```

```
while (condition) begin
    statements;
end
```

- loops: for and while

```
for (initialization; condition; update) begin
    statements;
end
```

Example

```

module mux4to1_if (
    input  I0, I1, I2, I3,
    input  S1, S0,
    output reg Y
);

always @(*) begin
    if (!S1 && !S0)
        Y = I0;
    else if (!S1 && S0)
        Y = I1;
    else if (S1 && !S0)
        Y = I2;
    else
        Y = I3;
end
endmodule

```

```

module mux4to1_case (
    input  I0, I1, I2, I3,
    input  [1:0] S,
    output reg Y
);

always @(*) begin
    case (S)
        2'b00: Y = I0;
        2'b01: Y = I1;
        2'b10: Y = I2;
        2'b11: Y = I3;
        default: Y = 1'bx;
    endcase
end
endmodule

```

BEHAVIORAL MODELING IN VERILOG

Behavioral modeling describes the **functionality (behavior)** of a digital circuit without describing **how it is implemented**.

1. WHAT IS BEHAVIORAL MODELING?

- Describes the circuit in terms of **algorithms** and logical behavior.
- Uses procedural statements inside **always** and **initial** blocks.
- More abstract than gate-level and dataflow modeling.
- Ideal for complex arithmetic, control logic, FSMs, and algorithmic blocks.
- Closely matches **RTL** thinking.

2. BASIC BUILDING BLOCKS

INITIAL BLOCK

- Executes **once at time 0**.
- Used for initialization.
- Not synthesizable in most cases (except for FPGA power-up initial values).

```
initial begin
    sum = 0;
    cout = 0;
end
```

ALWAYS BLOCK

- Executes whenever sensitivity list conditions are satisfied.
- Used to model combinational and sequential behavior.
- Synthesizable.

```
always @(*) begin
    // combinational logic
    sum = a + b;
    cout = carry_in + (a & b);
end
```

3. ALWAYS BLOCKS – TYPES (For Adders)

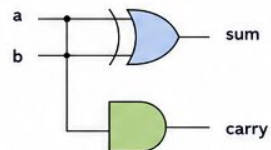
Type	Sensitivity List	Purpose	Example (Adder Context)
Combinational	always @(*) or always @(a or b or cin)	Pure combinational adder logic	<pre>always @(*) begin {cout, sum} = a + b + cin; end</pre>
Sequential (Clocked)	always @(posedge clk)	Registered / Pipelined adder	<pre>always @(posedge clk) begin {cout, sum} <= a + b + cin; end</pre>
Level-Sensitive (Latch)	always @(enable or a or b or cin)	Adder with enable (inferred latch)	<pre>always @(enable or a or b or cin) begin if (enable) {cout, sum} = a + b + cin; end</pre>



Use **non-blocking** assignment (<=) in **sequential logic** and **blocking** assignment (=) in **combinational logic**.

4. EXAMPLE 1 – HALF ADDER (BEHAVIORAL MODELING)

Logic:



Behavioral Model (Verilog):

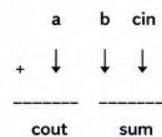
```
module half_adder_beh (
    input a, b,
    output sum, carry
);
    always @(*) begin
        sum = a ^ b; // sum
        carry = a & b; // carry
    end
endmodule
```

Explanation:

- always @(*)** ensures the block executes whenever a or b changes.
- Blocking** assignment (=) is used for combinational logic.
- This is a pure behavioral description (what the circuit does).

5. EXAMPLE 2 – FULL ADDER (BEHAVIORAL MODELING)

Logic / Function:



Equations:

```
sum = a ^ b ^ cin
cout = (a & b) | (b & cin) | (a & cin)
```

Behavioral Model (Verilog):

```
module full_adder_beh (
    input a, b, cin,
    output sum, cout
);
    always @(*) begin
        sum = a ^ b ^ cin;
        cout = (a & b) | (b & cin) | (a & cin);
    end
endmodule
```

Explanation:

- The functionality is described using behavioral statements.
- No gates or internal connections are mentioned.
- Synthesizable and easy to understand.

6. INITIAL vs ALWAYS

Feature	INITIAL	ALWAYS
Execution	Once at time 0	Whenever sensitivity list condition is met
Purpose	Initialization	Functionality / Behavior
Common Use	Set initial values (testbench, memories, counters, etc.)	Model combinational or sequential logic
Synthesizable	Not always	Yes (if written properly)

7. KEY TAKEAWAYS

- Behavioral modeling focuses on what the circuit does, not how it is built.
- Use **initial** block for one-time initialization.
- Use **always @(*)** for combinational adder logic.
- Use **always @(posedge clk)** for registered / pipelined adders.
- Blocking (=) for combinational, Non-blocking (<=) for sequential.
- Behavioral modeling is concise, readable and closely matches RTL.



Behavioral modeling using **initial** and **always** blocks provides a powerful and compact way to describe adders in Verilog while focusing only on **functionality**.

Textbooks

innovate

achieve

lead

